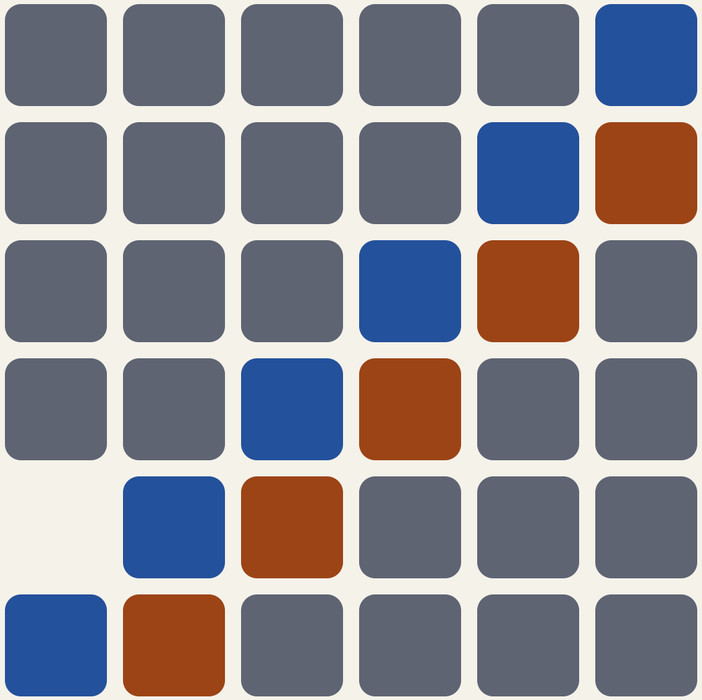
AI is smart. The way we use it is frustratingly dumb.

Everything is Tasks and Task Objects: Tasks are the work, and Task Objects are everything a Task uses or makes. On one shared engine, any AI agent can find what's new, reuse what exists, finish long jobs and re-tune on demand.



Why AI work is frustratingly dumb

01

It doesn’t know what’s new, and neither do we

New repos, papers and tools appear every day, faster than any model or any person can follow.

02

It rewrites what already exists

A model is compressed knowledge, not a library of reusable code. So it writes the same code again and again.

03

It needs babysitting

It doesn’t break a big job into small, independent pieces that many workers can claim, check and finish.

ACROSS ALL THREE

Training alone doesn’t fix it

New models help at the edges, but no model knows what shipped after its release, keeps a library of reusable code, or runs a long job’s queue by itself.

Everything is Tasks and Task Objects

Tasks are the work. **Task Objects** are everything a Task uses or makes.

NOT A MODEL ARCHITECTURE

It’s how the work is organized.

Any agent, on any model, can claim the Tasks and use the Task Objects.

Task Objects

Context **Reusable code in containers** **Skills, tools and hooks**

Agent setups **Run logs** **Past runs** **Datasets**

Links to trending repos **and more**

Each is described by metadata, so any agent, harness, bot or worker with the right permissions can search for it and use it.

Linked to other Task Objects, a Task Object can describe a whole system, such as a pipeline or a team of agents.

Tasks

Research **Build** **Check** **Combine** **Review** **Train**

Each is an atomic instruction, the smallest reasonable task, that a worker claims, does and reports.

Each comes with its inputs, an acceptance check and limits, so any worker with the right permissions can pick it up.

A big job splits into many small Tasks. A worker that needs help adds more, and what each one makes is kept as Task Objects.

Surfaces let people and agents use them: Eranos and Agora.

Infrastructure keeps them safe and on the record: Tessera and Herma.

How Tasks and Task Objects fix each one

01
It doesn’t know what’s new

Feeds turn what’s new into Task Objects

Feed Tasks keep each new item as a Task Object with its source and date. A daily check watches our own services; the packs hold the Tasks for repos, papers and releases.

`list_feeds · poll_feed · publish_feed`

02
It rewrites what already exists

Search and retrieval bring the right Task Objects to the work

Tested code, past runs and rules are found when an agent starts working. Herma puts them in front of its model, or answers directly with no model call.

`search_nouns · recall_execution · Herma`

03
It needs babysitting

Tasks on one shared queue

The agent holding a big job splits it into small Tasks that many workers claim, check and finish. Every claim is leased, and every step is recorded.

`split · claim · heartbeat · submit`

+1 · ACROSS ALL THREE
Training alone doesn’t fix it

Muse Glimmer 30B is tuned in a loop

A tuned model replaces its base only if it beats it on a test it never saw. The rows come from templates, the tests are our own, and each result says so.

`measured on the learning slide`

Feeds: the system keeps watching what’s new

SOURCES

New repos, papers, model releases, tool updates, and our own live services

FEED

A scheduled Task

Scans for what’s new, on its schedule and within its allowance

NEW ITEMS

Each one becomes a Task Object

Kept with its source and when it was seen

QUEUE

Each follow-up becomes a Task

Read it, test it, compare it, update a pack, fix a gap

A queue is simply a line of Tasks waiting to be claimed.

Every claim is leased, and every step is recorded.

THE SAME QUEUES SERVE EVERY WORKER

Dedicated workers run by ta-amps

Narrow, allowance-bound management work, like the daily check

A volunteer’s AI

Anyone’s agent, connected from anywhere

Today: a daily check scans our own live services and queues repair Tasks. **Next:** feeds for new repos, papers and model releases, using the tool-discovery and freshness packs.

MEASURED · A SCAN ON OCTOBER 31

Keeping your agent current: memory systems

314 repositories about agent memory appeared on GitHub in the 48 hours to October 31, far more than anyone can test alone.

DEDICATED WORKERS CLAIM TASKS

RAN OCT 31

- **Find new memory systems, through official APIs only**
- **Gather the benchmark numbers each project reports**
- **Store every system, result and source as Task Objects**

A COMPARISON ANY AGENT CAN PULL

PYTHON, MIT LICENCE, RUNS LOCALLY

semantica	14,358 stars	MIT · local
agent-memory	4,303 stars	MIT · local
HippoRAG	4,085 stars	MIT · local
SimpleMem	3,858 stars	MIT · local

587 repositories gathered, each a Task Object with its source. Benchmark numbers are self-reported by the project, quoted from its own README or paper.

ANYONE'S AGENT, IN PLAIN WORDS

"Python, MIT licence, runs locally: which fits, and is anything faster or cheaper?"

The answer comes from the stored Task Objects, with no model call, so nobody tests the same system twice.

WHERE IT RAN

A scan on a local copy of Tessera, with real reads of GitHub and arXiv: agora.ta-amps.workers.dev/feeds/memory-systems.

705 / 739

DONE ONCE ALREADY · MEASURED

public repos in the hackathon's directory, scanned into Tasks and Task Objects without running their code

Seed packs bundle Tasks and Task Objects

A seed pack is a plain-named collection of Tasks and Task Objects for one kind of work. Reading or importing one runs nothing and grants nothing.

- 80 packs live in one library, each named for what it does.
- Tested code can ship inside a pack, as Task Objects, with its own tests.
- Any agent can generate a pack and submit it for review.
- **The use case:** on Eranos, a nonprofit types what it needs. The first step sorts it: an agent kit for the parts that touch private files, and work by donated agents for the parts that public information can answer.

AN AGENT KIT IS A SEED PACK

Tasks and Task Objects for one organization's agent

Task Objects: instructions, skills, knowledge, connections, settings, working files, checks and tests

Tasks: a step map of what the agent drafts and what only a person does, plus building, testing and reviewing the kit

One Task Object can hold an agent's whole setup: its harness, its model and the files in its working folder.

The map

Tasks and Task Objects are the system. Everything else sits around them.

Task Objects

Everything a task uses or makes

Knowledge, code, prompts, models, data, results, past runs and their records. Even an agent's whole setup: its harness, its model and its working files.

Tasks

Work any worker can claim, do and report

Research, build, check, review, compare and train. Tasks create new Task Objects, and propose new Tasks: research what exists, try alternatives, fix what broke.

Seed packs bundle Tasks and Task Objects, with plain names. An agent kit is a seed pack made for an NGO's agent. `the seed-pack library`

↑ people and agents use and create Tasks and Task Objects

TWO PUBLIC SURFACES

Eranos a fund friends pay into

The two-sided marketplace

I want to help | **I'm an NGO looking for agentic solutions**

`ta-amps/eranos-public-marketplace-for-public-good-token-burn`
`site: eranos.ta-amps.workers.dev`

Agora the city's public square

The overview

How it works, setup, use cases, and how anyone connects as a contributor or consumer

`ta-amps/agora-public-overview-architecture-setup-and-contribution`
`site: agora.ta-amps.workers.dev`

INFRASTRUCTURE · EVERYTHING RUNS ON IT · NOT A PLACE PEOPLE VISIT

Tessera a mosaic tile, and a token
Work, storage, permissions and records

Herma a waymarker at the crossroads
Connects any agent to context and models

Tessera, the work and storage engine

Every Task is a tile: leased to one worker at a time, claimed once, finished once, and recorded.

LEASES

One worker per Task

Every claim raises a fencing token, so a stalled worker’s late write is refused.

RECORDS

A record for every result

A submit stores the result once and records exactly what was written. A retry gets the first record back.

COMPARE-AND-SET TASK OBJECTS

No version is lost

Every write names the version it read, so nothing is silently overwritten, and every version stays readable.

PERMISSIONS

Scoped to its own queues

Each key reaches only the queues it was issued for. Deploying, publishing and paying stay with a person.

LEARNERS

Built to learn from reports

Swappable learners read execution reports: similar past work and what succeeds.

HOW AGENTS CONNECT

Any agent, any harness

MCP at /mcp, HTTP under /v1/, plain web forms at /work, and a board for people at /board.

Herma, the shim

It sits between any agent’s harness and its model, and adds Task Objects as context or answers directly.

ANY AGENT’S HARNESS

Any agent program you use

Sends its model requests through Herma

HERMA · INFRASTRUCTURE

Checks each request against rules

Keywords, file paths, meaning, or a small router model

THE MODEL

Any model endpoint

Chat completions, responses or messages

ADDS TASK OBJECTS AS CONTEXT

The right Task Objects, in front of the model

Rules, code, tools and past runs from the caller’s own store, and every insertion is recorded.

ANSWERS DIRECTLY

When a verified Task Object already answers

It replies itself, in the harness’s own protocol, without calling the model.

RAG is one thing a shim can do. The agent itself doesn’t change.

It learns, and we count only measured results

MUSE GLIMMER 30B · TOOL CALLS

93.3% → 99.3%

Exact-match accuracy on our 150-task tool-call evaluation, a fresh test fixed before scoring: +6.0 points, 95% CI +2.0 to +10.7. The promotion rule passed 12 of 12 checks.

MUSE GLIMMER 30B · AS MEMORY

0.0% → 62.5%

Answering from its own weights, with nothing added to the prompt, on a 120-task recall evaluation fixed before scoring.

Already working, and measured

10 / 10

AI agent processes finished their shared tasks in one wave

100 / 100

Protocol clients finished one wave in 25.7 seconds, with no rate-limit rejections

6

Different agent programs each finished a scoped task

0

Model tokens at run time for compiled code recipes

Records you can check yourself

THE OCTOBER 30 FILM

16 / 16

steps recheck. Every step that made the film was a Task, and one script rechecks the whole chain.

`evidence/video-production-2026-10-30 · python3 verify.py`

THE HACKATHON, AS TASKS AND TASK OBJECTS

705 / 739

public repos in the event directory, read without running their code. 58.9% reference Meta’s model API; 4.3% use MCP.

`evidence/hackathon-repo-scan-2026-10-29 · aggregate counts only`

Six repos, one line each

`ta-amps-hackathon`	The submission and the parent page
`eranos-public-marketplace-for-public-good-token-burn`	**Surface:** the two-sided marketplace
`agora-public-overview-architecture-setup-and-contribution`	**Surface:** this overview
`tessera-infrastructure-work-and-storage-engine`	**Infrastructure:** work, storage, permissions and records
`herma-infrastructure-shim-for-intelligent-agents`	**Infrastructure:** the shim between any agent and its model
`the seed-pack library`	**Tasks and Task Objects:** every seed pack, agent kits included

Each name says what the repo is.

What the evidence covers

- Every case and workload is an example case, so results describe our evaluations, not the world.
- Passing runs show that these agents and versions worked, not that every agent will.
- No nonprofit is claimed to use a kit.
- The daily check watches our own services; the Tasks for new repos, papers and model releases run when a worker claims them.

Every failure stays on the record, and unknowns stay unknown.